

VISTAS (VIEWS) EN SQL SERVER

TABLAS VIRTUALES PARA CONSULTAS EFICIENTES



¿QUÉ ES UNA VISTA?



Una Vista (View) es una tabla virtual creada a partir de una consulta SQL.

- No almacena datos físicamente.
- Obtiene información de una o más tablas.
- Se actualiza automáticamente cuando cambian los datos originales.

¿PARA QUÉ SIRVE?

- ✓ Simplificar consultas complejas.
- ✓ Proteger información sensible.
- ✓ Reutilizar consultas frecuentes.
- ✓ Facilitar la generación de reportes.
- ✓ Mejorar la organización de la base de datos.



¿CUÁNDO UTILIZARLA?



Para generar reportes.



Para ocultar información sensible.



Para simplificar consultas repetitivas.



Para mostrar diferentes perspectivas de los mismos datos.

¿CÓMO FUNCIONA UNA VISTA?



TABLAS
(Datos reales)



CONSULTA SQL



VISTA
(Tabla virtual)



USUARIO
(Consulta la vista)

SINTAXIS BÁSICA

```
CREATE VIEW vw_ClientesActivos AS
SELECT IdCliente,
       Nombre,
       Ciudad
FROM Clientes
WHERE Estado = 'Activo';
```



USO DE LA VISTA

```
SELECT * FROM vw_ClientesActivos;
```



La vista se consulta como si fuera una tabla normal.



Tabla Clientes (Datos reales)

ID	Nombre	Estado
1	Juan	Activo
2	Ana	Inactivo
3	Pedro	Activo
4	Luis	Inactivo

❖ EJEMPLO PRÁCTICO ❖

Vista creada

```
SELECT *
FROM vw_ClientesActivos;
```



Resultado de la Vista

ID	Nombre
1	Juan
3	Pedro



❖ VENTAJAS ❖



Mayor seguridad



Consultas más simples



Reutilización de código



Fácil mantenimiento



Mejor organización de la información

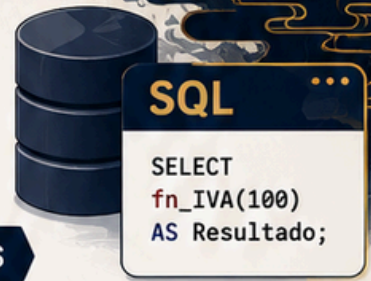


CONCLUSIÓN

Las Vistas (Views) permiten presentar información de forma **segura**, **organizada** y **eficiente**, facilitando el trabajo con bases de datos y optimizando las consultas SQL.



FUNCIONES EN SQL SERVER



BLOQUES DE CÓDIGO QUE GENERAN RESULTADOS



¿QUÉ SON LAS FUNCIONES?

Las funciones son objetos de base de datos que ejecutan una tarea específica y retornan un valor o una tabla como resultado.

- ✓ Permiten reutilizar código.
- ✓ Facilitan cálculos automáticos.
- ✓ Mejoran la productividad del desarrollo.



¿PARA QUÉ SIRVEN?

- ✓ Automatizar procesos repetitivos.
- ✓ Realizar cálculos complejos.
- ✓ Transformar datos.
- ✓ Validar información.
- ✓ Reducir errores en consultas.

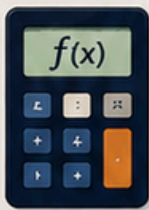


TIPOS DE FUNCIONES

1

FUNCIONES ESCALARES

Devuelven un único valor.



Ejemplos comunes:

- GETDATE()
- UPPER()
- LOWER()
- LEN()
- ROUND()

Ejemplo:

```
SELECT UPPER('upla');
```

Resultado:

UPLA

2

FUNCIONES DE TABLA

Devuelven una tabla completa.

Ejemplo:

```
CREATE FUNCTION fn_ClientesActivos()
RETURNS TABLE
AS
RETURN
(
    SELECT *
    FROM Clientes
    WHERE Estado = 'Activo'
)
```



¿CÓMO FUNCIONAN?



CREAR UNA FUNCIÓN ESCALAR

```
CREATE FUNCTION fn_IVA
(
    @Precio DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @Precio * 0.18
END
```



USO DE LA FUNCIÓN

```
SELECT dbo.fn_IVA(100);
```

Resultado:

18.00



EJEMPLO PRÁCTICO

Tabla Productos

Producto	Precio
Laptop	2500.00
Impresora	600.00

Aplicando la función:

```
SELECT dbo.fn_IVA(2500);
```

Resultado:

450.00



VENTAJAS



Reutilización
de código



Mayor
productividad



Menor cantidad
de errores



Fácil
mantenimiento



Consultas más
simples



APLICACIONES



Sistemas de ventas



Sistemas bancarios



Sistemas hospitalarios



Sistemas académicos



Sistemas empresariales



CONCLUSIÓN

Las funciones permiten automatizar operaciones dentro de SQL Server, mejorando la eficiencia, reutilización y organización del código en las bases de datos.



SCHEMA (ESQUEMAS) EN SQL SERVER

ORGANIZA Y PROTEGE TUS OBJETOS DE BASE DE DATOS



2

¿QUÉ ES UN SCHEMA?

Es la forma de organizar en un contenedor lógico que agrupa objetos dentro de una base de datos.

- ✓ Agrupar y administrar objetos relacionados.
- ✓ Puede contener múltiples tipos de objetos.
- ✓ Permite controlar permisos de acceso.
- ✓ Mejora la organización y el mantenimiento.



¿PARA QUÉ SIRVE?

- ✓ Organizar lógicamente los objetos de datos.
- ✓ Separar módulos o departamentos.
- ✓ Evitar conflictos de nombres de objetos.
- ✓ Controlar permisos de acceso.
- ✓ Mejorar la administración y escalabilidad.



OBJETOS QUE PUEDE CONTENER



Tablas

Almacenan los datos.



Vistas

Tablas virtuales basadas en consultas.



Funciones

Devuelven valores o tablas.



Procedimientos almacenados

Encapsulan operaciones SQL.

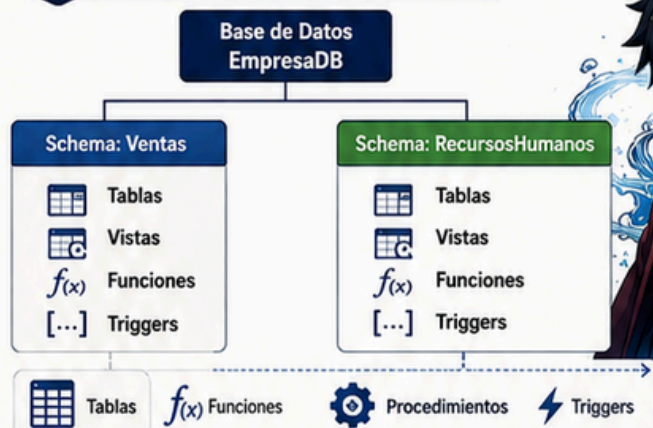


Triggers

Se ejecutan ante eventos específicos.



ESTRUCTURA DE EJEMPLO



CREAR UN SCHEMA

```
CREATE SCHEMA Ventas;
```



CREAR UNA TABLA DENTRO DEL SCHEMA

```
CREATE TABLE Ventas.Clientes
(
    IdCliente INT PRIMARY KEY,
    Nombre VARCHAR(100) NOT NULL,
    Ciudad VARCHAR(100)
);
```



VENTAJAS DEL USO DE SCHEMAS



ORGANIZACIÓN

Agrupar objetos de manera lógica.



SEGURIDAD

Controla y asigna permisos de acceso granulares.



ESCALABILIDAD

Facilita el crecimiento y mantenimiento de la base de datos.



ADMINISTRACIÓN

Simplifica la gestión de proyectos, módulos y usuarios.



BUEN USO Y CÓDIGO

Estándar para el manejo estructurado de objetos dentro de la empresa.



EJEMPLO PRÁCTICO

1 Crear esquema

```
CREATE SCHEMA Ventas;
```

2 Crear tabla en el esquema

```
CREATE TABLE Ventas.Clientes
(
    IdCliente INT,
    Nombre VARCHAR(100)
);
```

3 Consultar la tabla

```
SELECT *
FROM Ventas.Clientes;
```

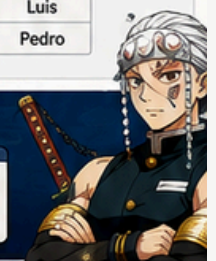
4 Resultado

IdCliente	Nombre
1	Ana
2	Luis
3	Pedro



CONCLUSIÓN

Los Schemas (esquemas) son fundamentales para organizar, proteger y optimizar la gestión de una base de datos, lo que aumenta la eficiencia operativa, el rendimiento y la seguridad en SQL Server.



ÍNDICES EN SQL SERVER



MEJOR RENDIMIENTO PARA TUS CONSULTAS

? ¿QUÉ SON LOS ÍNDICES?

Son estructuras especiales que permiten localizar registros rápidamente dentro de una tabla.



COMPARACIÓN SENCILLA

Un índice funciona como el índice de un libro: En lugar de revisar toda la información, permite encontrar rápidamente lo que necesitas.

🎯 ¿PARA QUÉ SIRVEN?

- ✓ Optimizar consultas.
- ✓ Reducir tiempo de búsqueda.
- ✓ Mejorar el rendimiento.
- ✓ Facilitar condiciones y búsquedas específicas.
- ✓ Mejorar la eficiencia del sistema.



MAYOR VELOCIDAD

Menos tiempo de respuesta



TIPOS DE ÍNDICES

1 CLUSTERED INDEX

Ordena físicamente los datos de la tabla.



Los datos se almacenan en el mismo orden del índice.

2 NON CLUSTERED INDEX

Ordena solo una copia separada de los datos.



El índice contiene punteros a las filas reales de la tabla.



⚙️ CREAR UN ÍNDICE

```
CREATE INDEX IX_Clientes_Nombre
ON Clientes(Nombre);
```



▶ USO DEL ÍNDICE

SQL Server utiliza índices automáticamente para analizar las consultas que incluyen columnas indexadas.



★ BENEFICIOS

- Mejor rendimiento general
- Reducción del tiempo de consulta
- Búsquedas más eficientes
- Mejor uso de recursos del sistema
- Consultas más optimizadas

⚠️ CONSIDERACIONES

- ✓ Generación de índices apropiados = UN BUEN RENDIMIENTO, EFICIENTE Y VÁLIDO.
- ⚠️ Requiere espacio en disco adicional.
- ⚠️ Deben usarse cuidadosamente para no afectar inserciones y actualizaciones.



EJEMPLO PRÁCTICO

1 Tabla Clientes

IDCliente	Nombre	Ciudad
1	Juan	Lima
2	Ana	Arequipa
3	Luis	Cusco
4	Pedro	Trujillo
5	Rosa	Iquitos

2 Crear índice en la columna

```
CREATE INDEX IX_Clientes_Ciudad
ON Clientes(Ciudad);
```

⚡ El índice se crea para acelerar búsquedas por ciudad.

3 Consulta utilizando el índice

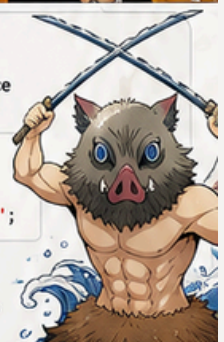
```
SELECT *
FROM Clientes
WHERE Ciudad = 'Lima';
```

⌚ ⌚ ⌚ ⌚ ⌚
Se ejecuta más rápido.

4 Consulta con índice (es más rápida)

```
SELECT *
FROM Clientes
WHERE Ciudad = 'Lima';
```

✓
Devuelve los registros rápidamente.



CONCLUSIÓN

Los índices son herramientas clave para mejorar el rendimiento de SQL Server. Su uso adecuado permite consultas más rápidas y eficientes, asegurando así una mejor experiencia y optimización en la gestión de datos.





UPLA
UNIVERSIDAD PERUANA LOS ANDES

STORED PROCEDURES (PROCEDIMIENTOS ALMACENADOS) EN SQL SERVER



AUTOMATIZA PROCESOS, MEJORA RENDIMIENTO Y SEGURIDAD



¿QUÉ SON?

Los Stored Procedures son programas almacenados en la base de datos que ejecutan una o varias instrucciones SQL.

Se almacenan en el servidor y pueden ser reutilizados.



¿PARA QUÉ SIRVEN?

- ✓ Automatizar procesos repetitivos.
- ✓ Centralizar la lógica de negocio.
- ✓ Mejorar la seguridad de los datos.
- ✓ Reducir el tráfico entre aplicación y servidor.
- ✓ Reutilizar código fácilmente.



VENTAJAS



Reutilización de código.



Mayor rendimiento.



Mayor seguridad.



Fácil mantenimiento.



Código más organizado.



¿CÓMO FUNCIONAN?



Aplicación
(o usuario)



```
EXEC
sp_MiProcedimiento
@param1, @param2
```

Llama al
procedimiento



SQL Server ejecuta
el procedimiento



Devuelve
resultados



SINTAXIS BÁSICA

```
CREATE PROCEDURE sp_ListarClientes
AS
BEGIN
    SELECT *
    FROM Clientes;
END
```



EJECUTAR PROCEDIMIENTO

```
EXEC sp_ListarClientes;
```



El resultado será el conjunto de registros devueltos por el procedimiento.

PROCEDIMIENTO CON PARÁMETRO

```
CREATE PROCEDURE sp_BuscarCliente
@Id INT
AS
BEGIN
    SELECT *
    FROM Clientes
    WHERE IdCliente = @Id;
END
```



EJECUCIÓN

```
EXEC sp_BuscarCliente @Id = 1;
```



Permite enviar valores al procedimiento para obtener resultados específicos.

EJEMPLO PRÁCTICO

1 TABLA CLIENTES

IdCliente	Nombre	Ciudad
1	Juan	Lima
2	Ana	Huancayo
3	Pedro	Lima
4	Luis	Ayacucho



2 CREAR PROCEDIMIENTO

```
CREATE PROCEDURE sp_ListarClientes
AS
BEGIN
    SELECT * FROM Clientes;
END
```



3 EJECUTAR

```
EXEC sp_ListarClientes;
```



4 RESULTADO

IdCliente	Nombre	Ciudad
1	Juan	Lima
2	Ana	Huancayo
3	Pedro	Lima
4	Luis	Ayacucho

APLICACIONES COMUNES



Sistemas de
ventas



Inventarios



Recursos
humanos



Sistemas
académicos



Generación
de reportes

CONCLUSIÓN



Los Stored Procedures son herramientas poderosas en SQL Server que permiten automatizar tareas, mejorar el rendimiento y proteger los datos. Su uso adecuado contribuye a sistemas más eficientes, seguros y fáciles de mantener.

